

From DCC-GARCH Validation to Full Stress Testing Framework

Last updated: 2026-02-23

This plan removes emojis and reflects the current project status and deliverables in the repository, including the interview package.

Project Phases

Phase 0: Data Gathering and Integration — Completed

Objective: Build a clean, reproducible dataset of market/macro stress indicators with coherent metadata, coverage checks, and merge traceability.

- Scope: ~52 stress indicators (rates, spreads, FX, commodities, equities, credit)
- Ingestion and config
- Scripts: `collect_industry_data.py` (FRED, Yahoo Finance), `config/stress_indicators_config.py`
- Raw/processed: `industry_data_raw.csv`, `stress_indicators_expanded.csv`
- Merge and QA
- Script: `merge_industry_data.py`
- Outputs: `industry_data_summary.json`, `merge_report.json` (value coverage, NaN treatment, unit harmonization)
- Crisis coverage analytics (for validation and narrative)
- Script: `data_pipeline/analyze_crisis_data.py`
- Deliverables: Coverage matrices and event-wise heatmaps used in presentation material
- Result: Reproducible, metadata-driven dataset and reports powering downstream volatility and correlation modeling

Phase 1: Yield Curve Modeling (NSS/DNSS) — Completed

Objective: Estimate country yield curves and DNSS betas suitable for stress testing; decide frequency and filter design.

- Baselines and frequency
- Quarterly creation reviewed; analysis recommends Monthly for stress workflows
- Daily estimates via Kalman Filter + Rauch–Tung–Stiebel (RTS) smoother for high-frequency diagnostics

- Implementation
- Scripts: `reestimate_dnss_monthly.py`, `estimate_dnss_kalman_daily.py`, `analyze_dnss_creation.py`
- Notebooks: `NSS_Core_Implementation.ipynb`, `NSS_Visualization.ipynb`, backups under `backups/`
- Key choices and rationale
- Monthly DNSS re-estimation balances robustness and run-time; daily Kalman used for monitoring
- Lambda handling and regime stability monitored; visualization confirms smooth term-structure dynamics
- Outputs
- DNSS parameter panels, beta series aligned with market factors; figures used in interview package

Phase 2: DCC-GARCH Validation and Covariance Matrix — Completed

Objective: Ensure DCC-GARCH produces reliable, time-varying covariance matrices for the 52 risk factors.

Status: Validation sufficient; proceed to Phase 2.

1.1 Basic DCC-GARCH fitting — 100% complete - Task: Fit DCC-GARCH to all 52 stress indicators (+ 20 DNSS betas) - Deliverable: Converged model with parameters (a, b) - Validation: $a + b < 1$ (stationarity), reasonable persistence - Files: `DCC GARCH MODEL/fit_dcc_garch.py` (executed) - Output: `DCC GARCH MODEL/results/dcc_garch_parameters.csv` - Timeline: Completed October 2025 - Results summary: - 72 series fitted - 100% convergence - Typical parameters $a=0.05$, $b=0.94$ ($a+b \approx 0.99$) - Daily coverage: ~9,320 observations (1998–2023)

1.2 Model diagnostics — sufficient - Tasks performed: - Out-of-sample forecast test: average improvement ~91.7% vs naive (1-day ahead) - Parameter stability: 0% explosive ($a+b \geq 1$), 76.4% stationary - Persistence half-life analysis documented - Positive definite and correlation bounds checks passed - Historical backtest: COVID detected ~7 days before crash; correct timing for 2008/2011 - Pending (optional): Ljung-Box, ARCH-LM - Deliverables: validation plots and reports - Files: `validate_model_robust.py`, `backtest_crisis_periods.py` - Outputs: `validation_results/`, `backtest_results/`

1.3 Time-varying correlation analysis — basic complete (optional deep dive) - Correlation time series saved and plotted - Focus pairs analysis can be added later (optional)

1.4 Regime-conditional correlations — not started (optional)

1.5 Out-of-sample forecasting — sufficient - Method: rolling 1-day ahead; compared to realized volatility; MAE as metric - Outputs: `validation_results/volatility_forecast_test.csv` - Multi-horizon (5d, 20d): not done (low priority)

Phase 2 deliverables summary - `DCC GARCH MODEL/results`: parameters, conditional volatilities, correlations, unconditional matrix, plots - `validation_results`: forecast tests, persistence, clustering, dashboards - `backtest_results`: crisis timing and summaries - Documentation: multiple MD reports under

Documentation/

Decision points (resolved) - Regime analysis: skip for now; DCC captures time-variation - Longer-horizon forecasts: optional; 1-day sufficient - Residual tests: optional; forecast test is stronger evidence

Public-finance block non-convergence

- Block fits for each country-level **public_finance** block (USA, Germany, France, Italy, Spain) ended with only one of the four GARCH series converging because the canonical rate/yield/spread series were missing. The absent instruments are recorded in DCC GARCH `MODEL/results/block_fit_metrics.json`—examples include `IRLTLT02{US, DE, FR, IT, ES}M156N`, `BOND_{United, Germany, France, Italy, Spain}_{10Y, 2Y}`, `TERM_SPREAD_{COUNTRY}_10Y_2Y`, `EUR3MTD156N`, and each country's `GC.DOD.TOTL.GD.ZS_{COUNTRY}` level. Without this coverage the persistence components chase long-memory decay instead of converging to a stationary solution, so forcing a full convergence would misrepresent the economic signal.
- Per FIGARCH / realized-volatility literature, persistent non-convergence can be acceptable as it reflects very slow decay rather than a technical failure. We therefore tolerate the partial fit provided the updates remain stable and positive definite; all diagnostics (residual Ljung-Box/ARCH-LM, correlation bounds, forecast MAE, etc.) stay loggable so the block can still signal instability. At the same time, we focus formal validation (plots, KPI dashboards, crisis backtests) on the fully converged blocks (macro, real_estate, financial_markets, and the miscellaneous block when its >80% series converge) while flagging the troubled blocks so downstream consumers know to treat their σ paths as informational only.

Data completion plan

- To close the data gap we plan to source and merge the missing series before rerunning the public-finance blocks:
- Extend `collect_industry_data.py` (endpoints/config handled in `industry_expansion_schema.json`) to fetch the missing BIS/IMF/ECB rates, sovereign bond yields, and spread metrics so they populate `industry_data_raw.csv` with metadata consistent with `config/country_blocks_extended.yaml`.
- Rerun `merge_industry_data.py` (and the coverage QA it writes to `industry_data_summary.json` plus `merge_report.json`) so the block assignment matrix can see the new series and attribute them to the public-finance blocks.
- Once the series exist in the merged panel, rerun the DCC-GARCH pipeline for the affected blocks, verify their residual/forecast diagnostics remain stable, and only then add them back into the validation summary. If the series cannot be obtained, document the proxy logic and keep the partial fits as flagged diagnostics, while validation continues to rely on the converged blocks.

Public-finance series backlog

- **BIS depository statistics (LBS)**
- Acquire bank-lending positions for the missing counterparty-country/sector combinations; focus on the large reporting banks (e.g., 5A all reporting countries) and split between domestic/cross-border so the public-finance block regulators gain exposure context. Combine quarterly levels (S: . . . :End of

period) with FX and break adjusted change series to cross-check reporting consistency.

- Key codes: Q:S:L:A:5A:A:5J:BOND_{EUR, USD, JPY} (where available), Q:S:L:A:T01:A:5J:A:5A:N for non-financial corporate lending to governments, and the currency-denominated L_DENOM variants to mirror domestic vs cross-border exposures.
- **IMF SDDS/IFS yields & spreads**
- Pull IRLTLT02{US, DE, FR, IT, ES}M156N (benchmark term structure), GC.DOD.TOTL.GD.ZS debt-to-GDP levels, and SPR{COUNTRY}10Y2Y spread series for each public-finance block country. These feed contextual leverage and duration diagnostics.
- **ECB SDW & Eurostat public debt**
- Include ECB sovereign bond yields for 2y/10y, EONIA/T/N funds rates, and Eurostat public-debt ratios (ESA2010). Align with L_MEASURE labels so metadata-driven block assignment can still rely on config/country_blocks_extended.yaml tagging rules (frequency, asset class, counterparty type).
- **Metadata expectations**
- Ensure every new series carries the standard metadata columns (Series, L_MEASURE, L_POSITION, L_INSTR, L_DENOM, L_CURR_TYPE, L_PARENT_CTY, L_REP_BANK_TYPE, L_REP_CTY, L_CP_SECTOR, L_CP_COUNTRY) so merge_industry_data.py can automatically map them into blocks, coverage checks, and analysis_outputs/block_membership_{ISO}.csv entries.
- Document downstream requirements (e.g., frequency alignment, units) inside industry_expansion_schema.json so config-driven ingest scripts continue to honor the planned coverage thresholds.

Pipeline extension checkpoints — IMPLEMENTED

- **Data ingestion & metadata intake**
- collect_industry_data.py now loads config/country_blocks_extended.yaml at startup to identify required vs optional series, tracks collection status for each category, and logs which optional series were skipped due to unavailable sources (BIS/IMF/ECB).
- Collection summary written to industry_data_summary.json includes collection_status section showing counts and lists of missing required/optional series.
- Script gracefully continues when optional series cannot be fetched, preventing empty data from entering the pipeline.
- **Coverage tracking & QA**
- merge_industry_data.py enhanced to cross-reference merged series against country_blocks_extended.yaml, automatically identifying which required/optional series are present in the final dataset.
- Merge report (merge_report.json) now includes optional_series_status section documenting coverage: required_present, required_missing, optional_present, optional_missing with full lists.
- New script scripts/check_block_coverage.py validates coverage per country block, calculates percentage coverage, and flags blocks with insufficient data (<70% required series). Output saved to analysis_outputs/block_coverage_report.json.

- **DCC reprocessing**
- Once the merged panel includes available series, rerun the relevant block fits via DCC GARCH MODEL/fit_dcc_garch.py, capturing the diagnostics outlined earlier (persistence, Ljung-Box, correlation bounds).
- Use scripts/check_block_coverage.py before running DCC to verify which blocks have sufficient coverage.
- Archive diagnostics under analysis_outputs/dcc_revalidation/public_finance_{YYYYMMDD}.json with versioned Sigma paths.
- If convergence fails, record the input series set (with metadata) and residual diagnostics so the plan can refer to why certain series remain excluded.

Updated next steps — OPERATIONAL

Run the enhanced pipeline: powershell python collect_industry_data.py python merge_industry_data.py python scripts/check_block_coverage.py This will automatically track which required/optional series are available and document coverage status.

Review coverage reports: - Check industry_data_summary.json for collection status (required vs optional) - Check merge_report.json for merged dataset coverage - Check analysis_outputs/block_coverage_report.json for per-block validation

Proceed with DCC-GARCH: - Only run DCC fits for blocks marked "✓ READY" or "■ PARTIAL" (≥70% coverage) - Skip blocks with "✗ INSUFFICIENT" status until more data becomes available - Use DCC GARCH MODEL/fit_dcc_garch.py with block-level filtering

Document gaps: - BIS/IMF/ECB series marked as optional in country_blocks_extended.yaml will be gracefully skipped - Missing optional series logged in reports but do not block pipeline execution - Any remaining gaps noted in analysis_outputs/diagnostics/public_finance_{date}.md

2.5 Alternative volatility and correlation models — integrated / extensible Purpose: Benchmark and extend baseline DCC-GARCH with richer short- and long-memory plus asymmetric and tail dependence structures.

Volatility layer (mean reversion & memory) - HAR-RV (Heterogeneous Autoregressive Realised Volatility) - Code: Volatility_MeanReversion/models/har_rv.py, pipeline Volatility_MeanReversion/run_pipeline.py (HAR sections), tuner scripts/tune_har.py - Features: daily / weekly / monthly realised volatility averages (data_prep.build_har_features) - Outputs: Output/har_correlations/dynamic_correlations_har.csv, per-asset har_forecasts_*.csv, har_metrics_*.json - Use: Captures multi-scale volatility dynamics improving short horizon forecast stability - FIGARCH (Fractionally Integrated GARCH) - Code: Volatility_MeanReversion/models/figarch.py - Long-memory variance; complements HAR for persistence - GJR / E-GARCH robustness - Script: DCC GARCH MODEL/gjr_egarch_robustness.py - Purpose: Test leverage/asymmetry and conditional variance responsiveness - Docs: DCC GARCH MODEL/FINAL_VALIDATION_GUIDE.md, docs/IGARCH_Professional_Practices.md

Correlation & dependence extensions - Conditional correlation variants:

`correlation_models/dynamic_correlation.py` - Copulas (tail dependence / asymmetric joint moves):

`correlation_models/copula_models.py` - Architecture references:

`correlation_models/COMPLETE_ARCHITECTURE.md`, `correlation_models/MODULE_EXPLANATIONS.md`

- ADCC vs DCC sensitivity: `analysis_outputs/dcc_vs_adcc_gap_summary.csv`

Visual benchmarking - Model performance panels / metrics:

`Volatility_MeanReversion/visualization/model_performance.py` (produces comparative PNGs)

and `tools/render_compare_panels.py` - Heatmap integration:

`Volatility_MeanReversion/visualization/event_heatmaps.py` (shared layout, regime/event overlays for HAR/FIGARCH vs DCC)

Key takeaway: HAR-RV + FIGARCH offer improved capture of multi-horizon and long-memory variance; asymmetric GARCH tests validate leverage effects; copulas enable tail risk correlation refinement beyond linear DCC.

2.6 Crisis correlation analysis — completed/available - Purpose: Quantify correlation amplification and structure across Normal/Stress/Crisis regimes and critical dates - Scripts (DCC results as inputs) - Critical-date heatmaps: DCC GARCH MODEL/`create_correlation_heatmaps.py` - Country/regime heatmaps: DCC GARCH MODEL/`create_country_heatmaps.py` - Regime matrices (Normal/Stress/Crisis): DCC GARCH MODEL/`create_regime_covariance_heatmaps.py`, `create_regime_heatmaps_v2.py` - Empirical scaling: DCC GARCH MODEL/`create_scaled_heatmaps.py` - Diagnostics bundle: DCC GARCH MODEL/`create_diagnostic_plots.py` - Outputs and evidence - DCC GARCH MODEL/`heatmaps/` and `heatmaps_final/` PNGs (per country and comparisons) - `5a_correlation_heatmaps_full.png`, `5b_correlation_heatmaps_betas.png` (readable NSS-beta focus) - Takeaway: Crisis correlations are materially higher than normal regimes; structure is economically interpretable

Phase 3: Stress Testing — Active (Scenario-First Orientation)

Objective: turn the cleaned risk-factor panel + country blocks into an auditable stress-testing engine (deterministic + historical replay + Monte Carlo) where every modeled series is explicitly defined in the country blocks.

What Phase 3 produces

- A **country-by-block factor set** (monthly + daily) that is consistent across the whole project.
- **Sparse mappings** (ElasticNet/Lasso-style) that connect block drivers to the target(s) used in stress testing.
- A **volatility + correlation layer** (univariate volatility models + DCC/ADCC) that yields time-varying σ_t .
- **Scenario artifacts** (inputs, outputs, diagnostics, logs) that are reproducible for a given run.

Key inputs (canonical)

- Catalog and metadata: [data_repository/catalog.csv](#), [config/series_metadata.yaml](#)
- Block definitions: [config/country_blocks_extended.yaml](#)

- Clean panel(s): [data/cleaned_monthly_panel.parquet](#) plus any derived daily panels under [analysis_outputs](#)

Step 0 — Ingest & merge indicators (foundation)

What it does - Downloads/collects raw series (FRED/Yahoo/BIS/IMF/ECB depending on the config), writes raw snapshots, and records which sources succeeded. - Merges all raw feeds into a single canonical panel and writes merge/coverage reports. - Runs block coverage checks so we know which blocks are READY / PARTIAL / INSUFFICIENT.

Where it happens - Core scripts - [collect_industry_data.py](#): ingestion; writes `industry_data_raw.csv`, `industry_data_summary.json`, and `analysis_outputs/diagnostics/collection_series_metadata_review.json`. - [merge_industry_data.py](#): merge + QA; writes `stress_indicators_expanded.csv` (canonical panel used by coverage checks) and `analysis_outputs/diagnostics/merged_series_metadata_review.json`. - [scripts/check_block_coverage.py](#): coverage vs [config/country_blocks_extended.yaml](#); writes `analysis_outputs/block_coverage_report.json`. - Wrappers - [SRESS TEST PIPELINE/0.1_industry_data_collector.py](#) - [SRESS TEST PIPELINE/0.2_industry_data_merger.py](#) - [SRESS TEST PIPELINE/0.3_block_coverage_checker.py](#)

Why it matters - Every downstream step assumes the merged panel is the single source of truth. - Missing/partial blocks are not “hand-waved”; they are explicitly tracked and gated.

Step 1 — Coverage contract + resampling governance

What it does - Builds the “data health” view: completeness, missingness patterns, resampling actions (e.g., monthly series step-held to daily), and sanity checks. - Creates/updates the coverage threshold configuration (the “coverage contract”) that later steps use as a gate.

Hard rule (monthly → daily policy) - Monthly / sparse series are **not interpolated by default**. - Default upsampling is **step-hold (LOCF / forward-fill only)**. - **Backfill is disabled by default** (no lookahead). - Any interpolation (and any backfill) must be explicitly opted-in via the coverage contract at [analysis_outputs/coverage_threshold_config.json](#) under `daily_upsampling_policy`.

Where it happens - Core scripts - [scripts/data_health_checks.py](#): writes global diagnostics under `analysis_outputs/diagnostics`. - [analysis_outputs/coverage_threshold_config.json](#): coverage contract consumed by coverage checks, the watchdog, and daily factor preparation. - Wrappers - [SRESS TEST PIPELINE/1.0_clean_monthly_panel.py](#) (builds `data/cleaned_monthly_panel.parquet` consumed by Step 1.1) - [SRESS TEST PIPELINE/1.1_data_health_diagnostics.py](#) - [SRESS TEST PIPELINE/1.2_coverage_threshold_optimizer.py](#) (writes the baseline threshold; the actual iterative optimizer runs later in Step 7.0)

Step 2 — Freeze governed country blocks (post-cutoff + harmonized)

What it does - Validates that every country block is well-formed (required fields present, no duplicate series, consistent naming). - Emits “block membership” artifacts used by factor prep and by correlation modeling. - Adds the latest Step 0.3 coverage status (READY/PARTIAL/INSUFFICIENT) to the frozen block artifacts when available. - Applies the **current coverage cutoff** from [analysis_outputs/coverage_threshold_config.json](#) (`series_threshold`) and then enforces **weakest-ISO**

harmonization per sub-block (degrade all ISOs to the minimum surviving driver count $\$K\$$ after the cutoff). - Writes a reconciliation report listing any drivers dropped **despite** passing the cutoff due to the harmonization step: [analysis_outputs/diagnostics/harmonization_drops_above_threshold.md](#).

Note - Cross-country derived series (e.g., sovereign spreads vs Germany) are constructed upstream during Step 0 merging (see [merge_industry_data.py](#)); Step 2 does not create these series, it validates that the block definitions reference the correct final series names.

Where it happens - Core scripts - [scripts/prepare_country_blocks.py](#) - Wrappers - [SRESS TEST PIPELINE/2.1_country_block_definer.py](#)

Outputs - [outputs/country_block_definition.json](#) -
[analysis_outputs/diagnostics/block_membership_matrix.csv](#) -
[analysis_outputs/diagnostics/country_block_metadata_warnings.md](#)

Downstream note - Steps 3+ prefer consuming the frozen [outputs/country_block_definition.json](#) so the applied cutoff + harmonized $\$K\$$ are actually respected by factor prep, mappings, and DCC/ADCC.

Step 3 — Build monthly/daily factor panels (transforms + PCA + FCI)

What it does - Applies transforms and standardization so block factors share a consistent scale and interpretation. - Builds lags, optional PCA components (when needed), and emits factor manifests so every derived column has an origin. - Constructs daily factor panels used by the volatility/correlation layers and by daily mapping/backtests.

Note - The upsampling policy is defined upstream in Step 1 (coverage contract) and enforced when building daily factor panels. - Literature-mode (block factors): when resampling the merged panel to -- literature-freq (default month-end), the panel is forward-filled after resample (step-hold) before applying the coverage/min-obs gate. This prevents quarterly series with complete native-frequency coverage from being incorrectly treated as sparse at monthly frequency.

How it works (monthly factors) - Input: monthly-aligned merged panel (stress indicators + DNSS betas) resampled to month-end. - Driver selection: for each ISO, take the governed driver allow-list from [outputs/country_block_definition.json](#) (fallback to [config/country_blocks_extended.yaml](#) only if the frozen artifact is missing). - Coverage gate: drivers must meet `series_threshold` from [analysis_outputs/coverage_threshold_config.json](#) (and must not be `do_not_use`). - Standardization: drivers are z-scored (mean 0, std 1) on the monthly panel.

How PCA is applied (monthly) - PCA is computed *within a block* only when the block appears highly collinear. - Current trigger: if any pairwise absolute correlation within the block exceeds 0.95 (and there is enough data to estimate correlations). - Components kept: minimum number of PCs to reach 90% cumulative explained variance, capped at 6 PCs. - Output: block PC columns are appended to the country factor table; loadings and explained-variance CSVs are written for audit.

How FCI is applied (monthly) - FCI is computed after standardization as a simple proxy index built from z-scored stress proxies. - Default: arithmetic mean of the available proxy columns (global proxies like VIX/credit spreads + any country-specific proxies present). - Optional mode: a PCA-based index can be used when enabled (first PC of the proxy set), but the default remains the mean for stability and interpretability.

How it works (daily factors) - Input: the daily panel from `stress_indicators_expanded.csv`. - Governance: the daily pipeline also prefers the frozen Step 2 artifact so the cutoff + harmonized \$K\$ are respected. - Resampling/fill: each block is filled using the Step 1 coverage contract `daily_upsampling_policy` (step-hold by default; interpolation/backfill are strictly opt-in). - Standardization: each ISO/block is scaled with a `StandardScaler`, and the fitted scaler is saved per ISO/block for reproducibility.

How PCA is applied (daily) - PCA is applied per block using heuristics to avoid unnecessary dimension reduction. - Trigger: PCA runs if (a) the mean absolute correlation is above a threshold (default 0.55) or (b) the block has many series (default trigger at 5 series), with a minimum of 3 series. - Components kept: enough to hit 90% cumulative variance, capped by both a max-components setting and an additional per-block cap (default: at most 2 PCs per block) to avoid "PCA swallowing the model". - Output: daily PCs are appended to the daily factor table; PCA component summaries and loadings are written under the daily factor output directory.

How FCI is applied (daily) - The daily FCI is built from the concatenated "financial*" blocks only. - Trigger: requires at least 3 underlying standardized financial series. - Method: 1-component PCA on the filled financial proxy set (first PC) to produce a single daily FCI series. - Output: written as `FCI_{ISO}_daily.csv` and also included in the daily factor table.

Hard rules (governance) - Step 3 should prefer the frozen Step 2 artifact at [outputs/country_block_definition.json](#) so the applied cutoff + harmonized \$K\$ are actually respected. - Driver coverage gating should use `series_threshold` from [analysis_outputs/coverage_threshold_config.json](#) (do not hard-code thresholds in Step 3). - Any series marked `do_not_use` in [catalog.csv](#) must be excluded from factor construction.

Where it happens - Core scripts - [scripts/prepare_country_factors.py](#) (monthly factors; uses frozen blocks + `series_threshold` + `do_not_use`) - [SRESS TEST PIPELINE/daily_factor_preparation.py](#) (daily factors; enforces `daily_upsampling_policy`) - Wrappers - [SRESS TEST PIPELINE/3.1_country_factor_preparer.py](#)

Legacy (kept for backwards compatibility; prefer `daily_factor_preparation.py`) - [scripts/build_daily_factors_for_st.py](#) - [scripts/run_daily_for_st_pipeline.py](#) - [SRESS TEST PIPELINE/3.2_daily_factors_builder.py](#) - [SRESS TEST PIPELINE/3.3_daily_panel_builder.py](#)

Outputs (where to look) - Monthly: - `analysis_outputs/factor_preparation/{ISO}_factors.csv` - `analysis_outputs/factor_preparation/{ISO}_pca_components.csv` - `analysis_outputs/factor_preparation/{ISO}_{block}_pca_loadings.csv` - `analysis_outputs/factor_preparation/{ISO}_{block}_factor_constituents.csv` (long-form constituent map per factor; used downstream for frequency inference + audit) - `analysis_outputs/factor_preparation/{ISO}_{block}_pca_explained.csv` - `analysis_outputs/factor_preparation/factor_preparation_summary.md` - Literature-mode (block factors; fewer, cleaner drivers): - Generated by [scripts/prepare_country_factors.py](#) with `--literature` - Combined panel for Step 7.2: - `analysis_outputs/literature_factors/block_factors.within_block.csv` - `analysis_outputs/literature_factors/block_factors.across_blocks.csv` - Factor-based block definitions for Step 7.2: - `analysis_outputs/literature_factors/country_block_definition.within_block.json` - `analysis_outputs/literature_factors/country_block_definition.across_blocks.json` - Per-ISO manifests and PCA audits: - `analysis_outputs/literature_factors/{ISO}_literature_manifest.json`

analysis_outputs/literature_factors/literature_pca_loadings.csv -
analysis_outputs/literature_factors/literature_pca_explained.csv - Daily: -
analysis_outputs/factor_preparation_daily/{ISO}_factors_daily.csv -
analysis_outputs/factor_preparation_daily/{ISO}_manifest_daily.json -
analysis_outputs/factor_preparation_daily/pca_components_summary_daily.csv -
analysis_outputs/factor_preparation_daily/pca_loadings_summary_daily.csv -
analysis_outputs/FCI_{ISO}_daily.csv (when generated)

Step 4 — Train sparse mappings (ElasticNet/Lasso) + diagnostics

What it does - Trains sparse mappings from governed Step 3 drivers → an explicit, fixed target set (monthly) and Rt targets (daily). - Enforces governance exclusions: any series flagged `do_not_use` in [catalog.csv](#) is removed from the feature set even if it appears in factor files. - Prevents target leakage: if a target appears in the feature matrix (including lagged forms), it is dropped as a feature. - Uses time-series-safe diagnostics (walk-forward/TimeSeriesSplit CV, circular-shift permutation test, moving-block bootstrap stability) and writes per-target artifacts.

Diagnostics detail (monthly) - Permutation test (`--permutation-trials`): refits many models on time-series-safe null targets (circular shifts) to estimate a p-value for “does this mapping beat chance?”. This is compute-heavy. - Output:

analysis_outputs/feature_contributions/permutation_test_{ISO}_{target}.json - Stability selection (`--stability-bootstraps`): refits the model many times on moving-block bootstrap samples and records how often each feature is selected (nonzero coefficient). This yields a selection-frequency signal Step 5 can use to prefer stable drivers. - Output:
analysis_outputs/feature_contributions/stability_{ISO}_{target}.csv

Operational note - These diagnostics are intentionally kept (they are informative), but they can dominate runtime. A practical workflow is to run Step 4 once with small trial counts during development, and periodically re-run with larger `--permutation-trials` / `--stability-bootstraps` for “final” diagnostics.

Target configuration (monthly) - Explicit targets are defined in [config/step4_targets.yaml](#) (global targets + optional per-ISO targets).

Where it happens - Core scripts - [scripts/step4_lasso_pipeline_daily.py](#) - Wrappers - [SRESS TEST PIPELINE/4.0_lasso_pipeline.py](#) - [SRESS TEST PIPELINE/4.1_lasso_mapping_trainer.py](#) - [SRESS TEST PIPELINE/4.2_lasso_mapping_daily_trainer.py](#)

Outputs (where to look) - Monthly summary per ISO:

analysis_outputs/feature_contributions_{ISO}.csv - Monthly diagnostics per target under analysis_outputs/feature_contributions/: - CV MSE path: {ISO}_lasso_cv.csv or {ISO}_enet_cv.csv - Permutation test: permutation_test_{ISO}_{target}.json - Stability

frequencies: stability_{ISO}_{target}.csv - Daily: - CV + coefficients:

analysis_outputs/feature_contributions_daily/{ISO}_{target}_*.csv - Diagnostics JSON: analysis_outputs/model_diagnostics_daily/{ISO}_{target}_diagnostics_daily.json

Step 5 — Select final driver shortlist (stability + collinearity)

What it does - Converts “many candidate drivers” into a small shortlist (8–12 per ISO) that is stable, interpretable, and not dominated by redundant collinear features. - Makes Step 4 “target mapping” usable

downstream by turning per-target Lasso/ElasticNet outputs into a single governed factor set.

Governed configuration - Step 5 thresholds/weights are governed in [SRESS TEST PIPELINE/step5_shortlist.yaml](#) so runs are comparable (selection size, scoring weights, pruning thresholds).

Design intent (targets) - Step 5 is *target-aware* via the Step 4 outputs it consumes: the shortlist should be the smallest driver set that explains the Step 4 target set. - The intended Step 4 targets are macro outcomes + term-structure state: - DNSS betas ($\beta_0, \beta_1, \beta_2$) - GDP, unemployment, CPI (e.g., Eurostat GDP/unemployment for DEU/FRA/ITA/ESP where available) - Targets are explicitly configured in [config/step4_targets.yaml](#).

How it works (monthly) - Input signals (from Step 4): - Coefficient magnitudes and “top contributions” from `analysis_outputs/feature_contributions_{ISO}.csv` - Stability-selection frequencies from `analysis_outputs/feature_contributions/stability_{ISO}_{target}.csv` (when Step 4 stability bootstraps are enabled) - Target coverage: how many targets a feature appears in - Feature universe: - Step 5 is explicitly consistent with Step 4’s feature source: it ranks/prunes only features that appear in Step 4 outputs (`analysis_outputs/feature_contributions_{ISO}.csv`), i.e., the same feature space Step 4 actually trained on and selected from. - Feature values for collinearity checks are loaded from the Step 3 outputs (PCA components / factor panels / optional FCI) under `analysis_outputs/factor_preparation/` and `analysis_outputs/FCI_{ISO}.csv`. - Ranking: - Weighted score blends contribution size, stability frequency, and target coverage. - Optional performance-aware signal: Step 5 can up-weight contributions from targets where Step 4 shows strong out-of-sample fit (e.g., using Step 4 `test_r2` to form an “OOS-weighted contribution” term). This is configured via [SRESS TEST PIPELINE/step5_shortlist.yaml](#). - If stability files are absent (e.g., Step 4 stability disabled for speed), the weights fall back so ranking remains meaningful. - Forced inclusions (to prevent pathological “all-one-theme” shortlists): - High-explained-variance PCA components (when per-block explained-variance reports exist) - FCI-like features - Step 4 top contributors - Collinearity pruning: - Prefer cluster-based pruning (correlation clusters): build correlation clusters (using an abs-correlation threshold) and keep one representative per cluster (prefer forced features, then higher-ranked features). This is more stable and easier to justify than repeated greedy pairwise drops. - Optional VIF pruning can be applied after clustering to address residual multivariate collinearity.

Outputs (monthly) - Shortlist table per ISO:

`analysis_outputs/feature_shortlist/factors_{ISO}.csv` - Audit manifest (selected + dropped + reasons + missing features): `analysis_outputs/feature_shortlist/manifest.json`

Where it happens - Core scripts - [SRESS TEST PIPELINE/step5_shortlist_collinearity.py](#) - [SRESS TEST PIPELINE/step5_shortlist_collinearity_daily.py](#) - Wrappers - [SRESS TEST PIPELINE/5.1_collinearity_shortlist.py](#) - [SRESS TEST PIPELINE/5.2_collinearity_shortlist_daily.py](#)

Step 6 — Daily scenario-ready vol + ADCC (σ_t, Σ_t) + audit logs

What it does - Runs the daily chain (daily factor prep → daily mapping → daily shortlist). - Fits volatility + correlation for each ISO using the *daily* shortlist and daily ISO return series, producing scenario-ready time series objects: - Univariate volatility per series (GARCH-family) → σ_t - DCC/ADCC correlation recursion → Σ_t - Covariance series via $\Sigma_t = D_t R_t D_t$ - Writes reproducible artifacts (CSV + model metadata JSON) so a run can be audited and re-run. - Also persists σ_t explicitly as `analysis_outputs/diag_corr_daily/{ISO}_Dt_daily.csv` for scenario scaling; you can regenerate only this artifact via `daily_adcc_prep.py --dt-only`.

Daily ADCC in literature factor-space (confirmed working) - Purpose: prove downstream daily ADCC/DCC can consume the Step 3 literature block factors ($\{ISO\}_{\{block\}_f1/f2}$) rather than governed daily shortlist series. - Important: Step 6 reads the *daily-expanded* literature panels `analysis_outputs/literature_factors/{ISO}_block_factors_{freq}_daily.csv`. If you change Step 3 literature factor construction (e.g., resample/coverage logic), rerun Step 3 with daily expansion first so these files refresh: - `python scripts/prepare_country_factors.py --literature --literature-freq M --literature-expand-to-daily --iso {ISO}` - Step 6 safety/audit behavior (literature mode): - **Preflight staleness gate:** if the daily-expanded literature panel is older than the corresponding resampled panel and/or manifest, Step 6 will refuse to run unless `--force` is provided. - **Provenance in metadata:** `models/adcc/{ISO}_daily_adcc.json` records the exact input file paths plus mtime and sha256 hashes (so you can prove which Step 3 artifacts were used). - How to run (multi-ISO batch; recommended to avoid stale artifacts): - `python "SRESS TEST PIPELINE/6.2_backtest_daily_runner.py" --factor-space literature --literature-freq M --force --isos ITA FRA DEU USA ESP` - What to check (proof of correct factor-space): - The metadata JSON under `models/adcc/{ISO}_daily_adcc.json` must contain `factor_space: "literature"` and a columns list dominated by $\{ISO\}_{*_f1/f2}$ plus `Rt_daily`. - Optional raw proxies are appended when present in `analysis_outputs/factor_preparation_daily/{ISO}_factors_daily.csv` (e.g., V2X for EU ISOs, VIXCLS for USA, plus SOFR_3m where available). - Primary artifacts written by Step 6 (daily): - `analysis_outputs/diag_corr_daily/{ISO}_standardized_residuals_daily.csv` - `analysis_outputs/diag_corr_daily/{ISO}_Dt_daily.csv` - `analysis_outputs/diag_corr_daily/{ISO}_Rt_daily_pairs.csv` - `analysis_outputs/diag_corr_daily/{ISO}_Sigma_daily_pairs.csv` - `analysis_outputs/diag_corr/{ISO}_Sigma_daily_corr.csv` - `models/adcc/{ISO}_daily_adcc.json`

Where it happens - Core scripts - [scripts/run_phase3_with_logging.py](#) - [SRESS TEST PIPELINE/daily_adcc_prep.py](#) - [SRESS TEST PIPELINE/iso_adcc_diagnostics.py](#) - Wrappers - [SRESS TEST PIPELINE/6.1_daily_chain_runner.py](#) - [SRESS TEST PIPELINE/6.2_backtest_daily_runner.py](#) - [SRESS TEST PIPELINE/6.3_iso_adcc_runner.py](#) - [SRESS TEST PIPELINE/6.4_verification_with_logging.py](#)

Step 7 — Guardrails & benchmarks (watchdog + volatility + global DCC/ADCC)

What it does - Step 7 is the *guardrails + benchmarking* layer around the main pipeline: - **7.0 Coverage watchdog:** runs the coverage-threshold optimizer and persists an updated `analysis_outputs/coverage_threshold_config.json` when diagnostics (persistence/min-eigen) indicate the threshold should be adjusted. - **7.1 Volatility mean-reversion benchmarks:** runs the `Volatility_MeanReversion/` benchmarking pipeline (HAR/FIGARCH/etc.) used for model comparison and long-memory diagnostics. - **7.2 Global DCC-GARCH training:** runs the global DCC-GARCH fit script (baseline Engle-style DCC) used for cross-checking and validation.

Step 7.0 watchdog behavior (what it actually does) - It runs the underlying threshold sweep/refinement in [scripts/coverage_threshold_optimizer.py](#), which repeatedly calls the unified ISO diagnostic runner (`SRESS TEST PIPELINE/iso_adcc_diagnostics.py`) at candidate coverage thresholds. - It reads the resulting diagnostics per ISO (persistence extracted from CLI output + min eigenvalue from `analysis_outputs/diag_corr/{ISO}_Sigma_eigenvalues.csv`) and writes a row-per-ISO summary to `analysis_outputs/coverage_optimizer/coverage_threshold_summary.csv`. - The wrapper [SRESS TEST PIPELINE/7.0_coverage_threshold_watchdog.py](#) then: - persists the chosen threshold back into [analysis_outputs/coverage_threshold_config.json](#) (as `threshold`, `series_threshold`, and `block_threshold` for compatibility), - retries a limited number of times (decreasing the threshold by a

fixed step) if diagnostics fail to meet configured floors.

Why the watchdog exists - Coverage thresholding is a *data governance* decision, but it also has *numerical stability* implications for Σ_t . - The watchdog is a safety loop to keep the “coverage contract” in a range where the fitted correlation/covariance objects are usable (eigenvalues don't collapse; correlation persistence isn't pathological).

Note - The *country/ISO scenario-ready* R_t and Σ_t time series used by the stress engine are produced in Step 6 (daily) and by `iso_adcc_diagnostics.py` (ISO-level diagnostics), not by the wrappers in Step 7.

Hard rule (Step 7.1 input semantics) - The mean-reversion volatility benchmarks should run on return-like / stationary inputs, not on lag-features or mixed raw levels by accident. - The Step 7.1 runner now: - drops lagged columns from factor manifests (keeps only base or $*_{lag0}$), and - enforces stationarity by transforming non-stationary series using an ADF test (default: first difference; configurable).

Hard rule (Step 7.2 dimensionality control) - If a country block has too many variables for stable DCC/ADCC estimation, Step 7.2 may reduce that block via PCA *within the block* before fitting. - This behavior is opt-in via config overrides; when used it writes `pca_loadings.csv`, `pca_explained_variance.csv`, and `dimensionality_control.json` into the block's results folder.

Literature mode (Step 7.2 input swap) - Goal: follow literature practice by estimating correlations on a small set of block factors (PCA/composites) instead of large raw-series blocks. - Step 7.2 can be pointed at the Step 3 literature artifacts via config overrides (or env): - Set `DCC_LITERATURE=1`, OR pass a JSON overrides file to Step 7.2 / `fit_dcc_garch.py`: - `dcc.literature.enabled: true` - `dcc.literature.panel_path: analysis_outputs/literature_factors/block_factors.<mode>.csv` - `dcc.literature.block_definitions_path: analysis_outputs/literature_factors/country_block_definition.<mode>.json` - `dcc.literature.skip_nss_join: true` (default) - Example overrides file shape: `json { "dcc": { "literature": { "enabled": true, "panel_path": "analysis_outputs/literature_factors/block_factors.within_block.csv", "block_definitions_path": "analysis_outputs/literature_factors/country_block_definition.within_block.json", "skip_nss_join": true }, "allow_k1_blocks": true } }` - $K=1$ blocks are treated as *completed* by running univariate GARCH only (no DCC/ADCC), so they still emit Σ_t paths and do not cause block-level shutdown.

Where it happens - Core scripts - [scripts/coverage_threshold_optimizer.py](#) (invoked by [SRESS TEST PIPELINE/7.0_coverage_threshold_watchdog.py](#)) - [Volatility_MeanReversion/run_pipeline.py](#) - [SRESS TEST PIPELINE/fit_dcc_garch.py](#) - Wrappers - [SRESS TEST PIPELINE/7.0_coverage_threshold_watchdog.py](#) - [SRESS TEST PIPELINE/7.1_volatility_mean_reversion_runner.py](#) - [SRESS TEST PIPELINE/7.2_dcc_garch_trainer.py](#)

Hard rule (series governance) - The governed country-block pipeline (Steps 2–6) uses the frozen block artifact and the coverage contract; it does not silently invent new factors.

Step 8 — Post-fit diagnostics and stress-test readiness gate

What it does - Consumes the Step 7.2 outputs under `DCC GARCH MODEL/results/` and produces a single consolidated diagnostics bundle used to decide whether the fitted correlation/volatility layer is

usable for stress testing. - Aggregates (per block): - DCC parameters (a, b, a+b) and ADCC gamma (plus guardrail flags if applicable) - Persistence / overfit signals (e.g., $a+b \geq 0.98$, a at upper bound) - GARCH per-series log-likelihoods and convergence rates (from `dcc_garch_parameters.csv`) - Unconditional correlation minimum eigenvalue (from each block's `unconditional_correlation_matrix.csv`) - Update success rates and SPD projection counts (from fit metrics) - Produces a **readiness judgment** per block (PASS/WARN/FAIL) plus an overall verdict for whether we can proceed to stress testing.

Where it happens - Wrapper - [SRESS TEST PIPELINE/8.0_postfit_model_diagnostics.py](#)

Outputs - Diagnostics bundle: -

`analysis_outputs/postfit_model_diagnostics/block_postfit_diagnostics.csv` -

`analysis_outputs/postfit_model_diagnostics/postfit_summary.json` -

`analysis_outputs/postfit_model_diagnostics/postfit_report.md`

Why it matters - Step 7.2 produces *models*; Step 8 decides whether they are stable enough to be used as inputs to stress scenarios. - This creates an explicit “go/no-go (or go-with-warnings)” gate before stress testing runs.

Note on step numbering - This new Step 8 sits immediately before stress testing. - Any “stress testing engine / scenario generation” work is therefore treated as **Step 9** going forward (conceptually), even if those wrappers are introduced later.

Step 9 — Scenario governance + run contract (cross-country) What it does - Freezes scenario inputs for a run: the ISO universe, Step 5 shortlists, and the latest usable volatility/correlation windows ($\$D_t$, R_t , $\backslash\Sigma_t\$$) produced by Step 6. - Enforces the Step 8 readiness gate: scenarios may run only for ISOs/blocks that are PASS (or WARN when explicitly allowed in config). - Defines the **run contract** used by all downstream scenario steps: - Scenario specification schema (horizon, frequency, shocked variables, path shape, scaling rules, seed, `run_id`) - Output directory contract under `analysis_outputs/scenarios/<run_id>/`

Hard rule (canonical shock space) - All scenario generation operates in **factor shock space** (return-like innovations consistent with the vol/corr layer). - “Decode to levels” is a reporting concern and must be explicit (never implicit).

Where it happens (provisional) - Shared utilities (module-style; imported by later steps) - `SRESS TEST PIPELINE/scenario_spec.py` - `SRESS TEST PIPELINE/scenario_io.py` - Wrappers - `SRESS TEST PIPELINE/9.0_scenario_governance.py`

Outputs (where to look) - Run manifest: `analysis_outputs/scenarios/<run_id>/manifest.json` -

Frozen inputs snapshot (ISO list, factor list, model window metadata):

`analysis_outputs/scenarios/<run_id>/inputs_snapshot.json`

Step 10 — Deterministic scenarios (cross-country narratives)

What it does - Creates narrative/regulatory-style scenarios as **explicit paths** on a small set of governed factors. - Supports cross-country overlays (e.g., “periphery spreads widen more than core”) while keeping the factor universe fixed to Step 5 shortlists.

Scenario primitives - Path shapes: step, ramp, hump, mean-reverting decay, and multi-phase “shock + partial recovery”. - Scaling: absolute units (when defensible) or standardized shock units (e.g., N-sigma)

using current/selected $\$D_{\{t_0\}}\$$.

Where it happens - Wrappers - SRESS TEST PIPELINE/10.0_deterministic_scenarios.py

Outputs (where to look) - Factor shock paths:

analysis_outputs/scenarios/<run_id>/deterministic/factor_shocks.csv - Scaling diagnostics (records whether $\$D_{\{t_0\}}\$$ was used):

analysis_outputs/scenarios/<run_id>/deterministic/scaling_diagnostics.csv - Scenario definition (human-readable):

analysis_outputs/scenarios/<run_id>/deterministic/scenario_definition.json

Step 10.1 — Macro narrative + explainability plots (IMF/FSAP-style overlay)

What it does - Produces an **explicit macro narrative** as quarterly deviations vs an implicit baseline (GDP growth, inflation, unemployment, policy rate, spreads, etc.) for communication and documentation. - Generates simple **PNG plots** that overlay countries so cross-country severity is immediately visible.

Design note - This is an *overlay layer* on top of factor-innovation scenarios (Step 10). It is intentionally labeled "stylized" unless/until calibrated to a country macro model. - IMF/FSAP semantics: narrative magnitudes are treated as **communication inputs**. They may be used to *infer* factor shocks in Step 10.2, but they are not automatically treated as hard constraints unless explicitly configured.

Where it happens - Wrappers - SRESS TEST PIPELINE/10.1_macro_narrative_and_plots.py - Config (optional) - SRESS TEST PIPELINE/scenario_macro_templates.yaml

Outputs (where to look) - Macro narrative paths (quarterly deltas):

analysis_outputs/scenarios/<run_id>/deterministic/macro_narrative_paths.csv - Macro narrative levels (baseline + stressed, when baseline levels are provided):

analysis_outputs/scenarios/<run_id>/deterministic/macro_narrative_levels.csv - Macro narrative definition (semantics + severity method):

analysis_outputs/scenarios/<run_id>/deterministic/macro_narrative_definition.json -

Explainability plots: analysis_outputs/scenarios/<run_id>/deterministic/plots/

Step 10.2 — Factor shocks from macro narrative (ridge inversion; auditable IMF/FSAP mode)

What it does - Maps a small set of macro narrative deltas (e.g., CPI YoY, unemployment level) into the **canonical factor shock space** used by the stress engine. - Solves an inverse problem per ISO: choose factor shocks $\$z\$$ (in standardized factor space) such that the Step 4 macro-target mappings imply macro deltas close to the narrative. - Produces ISO-level diagnostics that distinguish: - **consistency** (corr/sign match) vs - **magnitude fit** (RMSE), and - **plausibility** (shock size / concentration).

Hard rules (governance) - The inversion target set must be **high-confidence**: targets with weak out-of-sample fit are not suitable as constraints. - Current guardrail: GDP growth is **disallowed** as an inversion target (may remain narrative-only / projection-only). - Canonical shock space is enforced in the inversion **factor** universe: - Lag-features (e.g., $\ast_{\text{lag1}}$) are excluded. - Structural ratios/levels that are not shockable innovations (e.g., debt-to-GDP level series like $\text{GC.DOD.TOTL.GD.ZS}_\ast$) are excluded. - If the filter removes all usable factors for an ISO, the ISO is marked **unsolved** for Step 10.2 (rather than silently inverting on non-canonical factors). - Deterministic chapter is deterministic-only: no Monte Carlo sampling in Step 10.*.

Lambda (λ) selection (defensible + auditable) - Default is an **auto- λ grid search** per ISO with two modes: 1) **Constrained fit**: pick the *largest* λ that still meets a relative fit tolerance (to avoid overfitting / extreme shocks). 2) **Scalar fallback**: if no λ meets the fit tolerance, minimize a single objective = fit loss + α ·plausibility penalty (soft cap on shock sizes). - Selection and grid results are written as CSV audit artifacts.

Plausibility constraint (z_{cap}) - In constrained mode, z_{cap} is treated as a **binding** constraint when feasible (i.e., reject λ that imply $\max |z| > z_{\text{cap}}$ if any feasible λ exist). - Additionally, Step 10.2 clips any emitted quarter-z values beyond z_{cap} (disable by setting $z_{\text{cap}} \leq 0$).

Where it happens - Wrapper - SRESS TEST PIPELINE/10.2_factor_shocks_from_macro.py - Config (templates) - SRESS TEST PIPELINE/scenario_macro_templates.yaml - SRESS TEST PIPELINE/scenario_macro_templates_fsap.yaml

Outputs (where to look) - Macro paths used for inversion:

analysis_outputs/scenarios/<run_id>/deterministic/macro_narrative_paths_used_for_inversion_<macro_tag>.csv

- Factor shocks inferred from macro:

analysis_outputs/scenarios/<run_id>/deterministic/factor_shocks_from_macro_<macro_tag>.csv

- Mapping + status:

analysis_outputs/scenarios/<run_id>/deterministic/factor_shocks_from_macro_mapping_<macro_tag>.csv

analysis_outputs/scenarios/<run_id>/deterministic/factor_shocks_from_macro_status_<macro_tag>.csv

- Diagnostics JSON:

analysis_outputs/scenarios/<run_id>/deterministic/factor_shocks_from_macro_diagnostics_<macro_tag>.json

- Auto- λ audit (when enabled):

analysis_outputs/scenarios/<run_id>/deterministic/factor_shocks_from_macro_auto_lam_grid_<macro_tag>.csv

-

analysis_outputs/scenarios/<run_id>/deterministic/factor_shocks_from_macro_auto_lam_selection_<macro_tag>.csv

Step 10.3 — Explainable deterministic scenario report (tag-aware; diagnostic-first)

What it does - Produces a deterministic, explainable report that ties together: - the factor shock scenarios (Step 10.0), - the macro narrative (Step 10.1), - the ridge inversion outputs (Step 10.2), and - the Step 4 mapping quality (train/test R^2 , transforms, sample sizes). - Emphasizes supervisory-style *diagnostics* over black-box scoring: - narrative-vs-implied macro consistency (corr/RMSE/sign match), - scenario severity indices (scenario-varying; not constant by construction), - systemic co-movement ("correlations go to 1" diagnostics), - driver→target interaction weights computed using the **effective per-ISO λ** chosen in Step 10.2. - Explicitly surfaces **low-confidence targets** (negative test R^2) as interpretation cautions.

Where it happens - Wrapper - SRESS TEST PIPELINE/10.3_explainable_scenario_report.py

Outputs (where to look) - Tag-aware report folder:

analysis_outputs/scenarios/<run_id>/reports/<macro_tag>/ - Markdown report:

explainable_report.md - Diagnostics CSVs: macro_target_consistency.csv,

severity_index_by_scenario.csv, target_comovement_summary.csv, etc. - Plots: plots/

Step 11 - Historical replay (episode library, cross-country)

What it does - Baseline mode (rotation OFF): **Filtered Historical Simulation (FHS)-style replay**. - Fit volatility model → compute standardized residuals z_t → replay an episode window of z_t . - Optionally re-volatilize with current conditional volatilities σ_{t_0} so the episode is expressed in the **current volatility regime**. - Runs in standardized-residual space (post-GARCH) to avoid mixed-frequency artifacts. - Optional advanced mode (rotation ON): rotate replay shocks into the current correlation regime (regularized) before re-volatilizing.

Design note (cross-country comparability) - Episodes are defined as: date window (+ optional ISO participation) + availability rules (minimum overlap).

Where it happens - Config - config/historical_episodes.yaml - Wrapper - SRESS TEST PIPELINE/11.1_historical_replay.py - Plots (literature-style bundle) - SRESS TEST PIPELINE/11.2_fhs_historical_replay_plots.py - Output folder: SRESS TEST PIPELINE/FHS Historical Replay Plots/<replay_run_id>/ - Companion docs (auto-written each time plots are produced): - SRESS TEST PIPELINE/FHS Historical Replay Plots/<replay_run_id>/README.md (explains plot categories + economic interpretation) - SRESS TEST PIPELINE/FHS Historical Replay Plots/<replay_run_id>/<episode_id>/README.md (episode-specific reading order + pointers)

Outputs (where to look) - Run folder: analysis_outputs/historical_replay/replay_<timestamp>/ - manifest.json (inputs + settings + blocks) - episodes/<episode_id>/block_z_shocks/<block_id>.csv (replayed shocks in z-space) - episodes/<episode_id>/block_innovations/<block_id>.csv (replayed innovations in unit space if re-volatilized) - episodes/<episode_id>/episode_summary.csv + plausibility_report.md - episodes/<episode_id>/episode_diagnostics.json (flags unusually large z-shocks and their drivers)

Default settings (stability) - rotate_to_current_correlation defaults to false (baseline FHS-style replay; avoids numerical amplification from whitening/rotation). - If rotation is enabled, use corr_shrinkage_lambda, corr_eig_floor, and winsor_abs_quantile to keep the replay well-conditioned. - Episode slicing prunes sparse series first (min_col_coverage) and then applies complete-case dropping to the remaining series.

Integration option - If you want historical replay outputs to live under an existing Step 9/10 scenario run, pass --scenario-run-id <run_id> (or --use-latest-scenario-run) so outputs land in analysis_outputs/scenarios/<run_id>/historical_replay/.

Step 12 — Stochastic Monte Carlo scenarios (distributional cross-country stress)

What it does

Core goal - Generate many plausible **joint scenario paths** for governed factors (and/or block factors) that are consistent with the fitted volatility/correlation layer, while respecting the project's frequency policy (no synthetic daily macro paths).

Frequency policy (applies here) - **Low-frequency macro/economic series** (monthly/quarterly): simulated on the native grid. - **Daily market/financial series**: simulated on the daily grid. - When a daily-aligned table is needed for downstream joins, low-frequency series are aligned using **step-hold / LOCF** (no interpolation; no backfill).

Outputs produced (cross-country stress, not trading VaR) - Tail percentiles of macro/market targets by ISO - Joint-stress metrics (e.g., probability that $\geq k$ countries breach a threshold) - Representative “systemic” draws saved as scenarios

Proposed Monte Carlo design (mixed-frequency; literature-aligned) 1) **Daily financial path simulation** - Simulate daily factor shocks in standardized space using the fitted dependence model (e.g., DCC/ADCC time-varying Σ_t or block-level Σ_t). - Sampling family options: - Multivariate Normal baseline - Student-t (fat tails) - Optional residual/bootstrap variants for robustness

2) **Monthly/quarterly macro path simulation (native frequency)** - Simulate macro shocks on the monthly/quarterly grid. - Minimal viable dynamics (no high-frequency indicators): block-wise AR(1) / VAR(1) in standardized space, with Student-t innovations if tails are required. - Cross-block dependence can be modeled at the macro grid using a correlation matrix estimated from the low-frequency panel (or a shrinkage estimator when sample size is small).

3) **Scenario assembly for propagation/reporting** - Write two aligned scenario products: - `daily_draws.csv`: daily series + macro carried forward (LOCF) - `macro_monthly_draws.csv`: the true native-frequency macro draws (authoritative) - Propagation steps must consume the authoritative grid per driver (monthly macro when relevant), and only use LOCF alignment when a model explicitly requires daily inputs.

Why this matches literature constraints - Avoids inventing within-month macro volatility without indicators. - Still produces daily scenario paths for market factors and a consistent macro state sequence for cross-country storytelling and target propagation.

Hard rule (reproducibility) - All stochastic runs must persist the RNG seed, sampling family, and any fitted tail parameters.

Implemented governance updates (current) - Low-frequency classification supports metadata-first logic via `config/series_metadata.yaml` plus per-factor constituent sources: - `analysis_outputs/factor_preparation/{ISO}_{block}_pca_loadings.csv` - `analysis_outputs/factor_preparation/{ISO}_{block}_factor_constituents.csv` (fallback when PCA loadings are missing) - Metadata coverage audit: `tools/audit_series_metadata_coverage.py` validates that every constituent series in either source has a frequency label. - Strict metadata mode: Monte Carlo can be run with **no Dt fallback** for classification (`--lowfreq-classifier metadata --lowfreq-metadata-strict`). - Diagnostics now include: - `diagnostics/lowfreq_gating_variance.csv` (ungated vs gated variance + zero-mass checks) - `diagnostics/lowfreq_update_calendar_validation.csv` (forward update schedule vs implied frequency) - Cost control + provenance: output manifests include a signature over args + key input hashes and can skip recomputation when unchanged. - Innovation backtest gate script: `tools/innovation_backtest_gate.py` writes per-factor distribution checks under the run's `diagnostics/`.

Known limitation (remaining Dt usage) - Forward **update-date scheduling** (for gating) is now governed by an explicit release calendar config (`config/release_calendar.yaml`) rather than projected from `Dt_daily`. - Remaining governance option: refine the calendar to be (a) holiday-aware and/or (b) series-specific where real release dates materially differ from the defaults.

Where it happens (provisional) - Wrappers - `SRESS TEST PIPELINE/12.0_monte_carlo_scenarios.py` - Plotting - `SRESS TEST PIPELINE/12.1_monte_carlo_scenario_plots.py` (writes `SRESS TEST PIPELINE/MC scenario plots/<run_id>/`)

Outputs (where to look) - Monte Carlo draws (optionally compressed/partitioned): - `analysis_outputs/scenarios/<run_id>/monte_carlo/daily_draws.csv` (small runs) - `analysis_outputs/scenarios/<run_id>/monte_carlo/daily_draws/` (large runs; per-ISO, per

draw-chunk shards; CSV/Parquet depending on availability) -
 analysis_outputs/scenarios/<run_id>/monte_carlo/macro_monthly_draws.csv (native-frequency macro shocks; includes month and update_date) - Summary percentiles + joint stress metrics:
 analysis_outputs/scenarios/<run_id>/monte_carlo/summary.json - Selected representative scenarios: analysis_outputs/scenarios/<run_id>/monte_carlo/representatives/ - Audit bundle:
 - analysis_outputs/scenarios/<run_id>/monte_carlo/manifest.json (seed, family, degrees-of-freedom if t, grids used) -
 analysis_outputs/scenarios/<run_id>/monte_carlo/diagnostics/ (marginal checks, correlation sanity, tail checks)

Plot interpretation note (units vs standardized) - Step 12 daily_draws shocks are written in **innovation units** (scaled by $\Delta t(t_0) / \text{vol}_{t_0}$). - For visualization and cross-series aggregation: - Per-factor plots may be shown in **innovation units** (more interpretable). - Block aggregates should be computed in **standardized space** (z-like) to avoid averaging mixed units (bps vs % vs levels).

Backup (in case of worsening): Practical solutions - Best fix (economically correct): make sure the missing block drivers actually survive into the MC factor list (upstream in factor preparation / harmonization / selection). If key series like EURIBOR_3m, Bank_equity_index_*, HPI / mortgage rates are being dropped before MC, the affected blocks become under-identified and can collapse into one shared proxy. - Second-best fix (prevents double-counting): ensure a series (and its lags) belongs to only one block for aggregation (choose a primary block or explicitly remove shared series like BIS_LBS_Household_Loans_* from one of the two blocks). This avoids “same factor counted twice” artifacts, but it is still a band-aid if the real issue is missing drivers. - Reporting fix: when a block has present_expected=1, flag it as under-identified and either (a) exclude it from comovement ranking, or (b) keep it but clearly annotate it in the regime/comovement markdown.

Backup (in case of worsening): Suggested improvements (beyond what's implemented) - Add a “coverage health” banner per ISO/block in the regime markdown (e.g., present_expected=1/5) so readers immediately see when a block is thin. - Add rank-stability / uncertainty for comovement: - Correlations: Spearman + bootstrap CI (rather than only point estimates). - Severity shares: q10/q90 are useful; adding q25/q75 (IQR) typically reads better. - Consider defining regimes using one consistent rule: either “All ISOs combined” regimes only, or explicitly state that per-ISO regime cutoffs are ISO-local (both are valid, but mixing them without labeling is easy to misread).

Step 13 — Reverse stress testing (goal-seeking, cross-country)

What it does - Solves for the **minimal adverse factor shock** that achieves a defined stress condition. - Supports cross-country constraints (single ISO threshold, or multi-ISO joint thresholds).

Optimization framing - Objective: minimize shock size under a metric (e.g., quadratic form using $\Sigma_{t_0}^{-1}$ or a diagonal proxy). - Constraints: sign constraints, plausibility bounds, target threshold(s) in propagated space.

Where it happens (provisional) - Wrappers - SRESS TEST
 PIPELINE/13.0_reverse_stress_scenarios.py

Outputs (where to look) - Reverse stress solution:
 analysis_outputs/scenarios/<run_id>/reverse_stress/solution.json - Sensitivity diagnostics:
 analysis_outputs/scenarios/<run_id>/reverse_stress/sensitivity.json

Step 14 — Scenario propagation to targets (and optional satellites)

What it does - Propagates scenario factor shocks to configured targets using the Step 4 mappings (and Step 5 shortlists). - Ensures consistent handling of standardized feature space vs raw units when producing target impacts.

Optional satellites (future) - Banking / sovereign / funding “satellite models” can be added later as a separate sub-step once the core propagation is stable.

Where it happens (provisional) - Wrappers - SRESS TEST PIPELINE/14.0_scenario_propagation.py - SRESS TEST PIPELINE/14.1_satellite_models.py (optional; future)

Outputs (where to look) - Target impacts:

analysis_outputs/scenarios/<run_id>/propagation/target_impacts.csv - Propagation diagnostics (missingness, clipping, unit decode):

analysis_outputs/scenarios/<run_id>/propagation/propagation_diagnostics.json

Step 15 — Scenario validation + comparability checks (acceptance tests)

What it does - Runs acceptance tests so scenario outputs are usable and comparable across countries and run IDs.

Validation bundle (cross-country) - Deterministic: plausibility + monotonicity/bounds checks + cross-country severity ordering checks. - Historical replay: “does it look like the episode?” sanity checks + scaling reasonableness. - Monte Carlo: stability of percentiles/joint-stress metrics across seeds; dependence sanity. - Reverse stress: verify constraints are met and the solution is locally minimal under the chosen metric.

Where it happens (provisional) - Wrappers - SRESS TEST PIPELINE/15.0_scenario_validation.py

Outputs (where to look) - Validation report:

analysis_outputs/scenarios/<run_id>/validation/report.md - Machine-readable validation summary: analysis_outputs/scenarios/<run_id>/validation/summary.json

Implementation note (script naming + placement) - All new scenario step wrappers should live under SRESS TEST PIPELINE/ and follow the existing numeric-prefix convention (e.g., 12.0_monte_carlo_scenarios.py). - Any shared scenario helpers (schema, IO, math utilities) should also live under SRESS TEST PIPELINE/ as importable modules (e.g., scenario_spec.py) so the plan remains consistent with the pipeline layout.

Note — Frequency policy (no high-frequency indicators)

Context - Some macro/economic series are published at monthly/quarterly frequency (e.g., CPI, unemployment, certain balance-sheet aggregates) and we do not have trusted high-frequency indicators to infer within-period dynamics.

Policy (model at native frequency) - **Model/simulate low-frequency macro series at their native frequency** (monthly/quarterly) rather than inventing daily paths via interpolation or synthetic noise. - When daily alignment is required for joins/plots/tables, use **step-hold / LOCF (forward-fill only; no backfill)**. - Treat impacts on daily financial variables primarily as: - **information shocks** around release dates (what becomes known), and/or - **macro state updates** at monthly boundaries (when the macro

state is refreshed).

Implications for later steps - Step 11 (historical replay): keep replay mechanics in z-space, but plots should avoid misleading one-day “release spikes” by presenting low-frequency series in a step/level-style display (without changing modeling outputs). - Step 12 (Monte Carlo): sample low-frequency macro drivers on the monthly grid; if daily paths are needed for downstream propagation, they should be derived from the monthly draws using step-hold alignment (not interpolation).

Scenario artifacts (where to look)

- Scenario runs and diagnostics live under [analysis_outputs/scenarios](#) (run_id folders) plus [logs](#).
- A run is considered complete when it contains:
- `manifest.json` (inputs + config + provenance)
- one or more scenario subfolders (`deterministic/`, `historical/`, `monte_carlo/`, `reverse_stress/`)
- `propagation/` outputs (if Step 14 is enabled)
- `validation/` outputs (Step 15)

Older version

Phase 3: Stress Testing — Active (Scenario-First Orientation)

Objective: Build an operational stress-testing capability driven by scenario generation, keeping data preparation, factor selection, volatility modeling, and reporting aligned with each mode of stress testing (deterministic, stochastic, historical replay, reverse stress).

Steps:

Metadata & ingestion setup - Document source/frequency/transform/coverage metadata in `config/series_metadata.yaml` and track ingestion scripts for public indicators (IMF FSI, ECB aggregates). - Ensure `regression_long.csv` and `country_blocks.yaml` align with that metadata to support reproducible pipelines.

Clean panel creation with QC - Resample to monthly using metadata-defined rules (business-day month end, monthly average as appropriate), keep quarterly series, apply seasonal adjustments when flagged, and transform per rule (`pct_change/log/diff`). - Enforce $\geq 70\%$ non-NaN coverage, log gaps to `outputs/missing_series.log`, and emit `data/cleaned_monthly_panel.parquet` plus a QC summary JSON.

Country-block definition & sub-block tagging - Formalize a `CountryBlock` schema that covers Macro, Markets, Banking, and External sub-blocks plus manual extras. - Export `outputs/country_block_definition.json` so every downstream consumer uses the same block structure.

Factor preparation loop - For each country, pool drivers via metadata, keywords, and `extra_series`, standardize (z-score), and generate flagged lags (0–3 months) restricted by coverage

and collinearity checks. - Apply block-level PCA when sub-block members are collinear (retain components explaining 85–90% variance, cap at $\min(\text{floor}(n_{\text{obs}}/10), 6)$) and record loadings in `outputs/pca_components_{country}.csv`. - Construct an optional Financial Conditions Index (FCI) from stress proxies (spreads, TED, VIX, banking equity) and save `outputs/FCI_{country}.csv`.

Lasso mappings to targets - Train `StandardScaler` → `LassoCV(TimeSeriesSplit, alphas=logspace(-6, 0, 50))` per target, persist models to `models/lasso_{country}_{series}.joblib`, and save coefficients, train/test R^2 , contribution rankings, and instability flags to `outputs/feature_contributions_{country}.csv`.

Factor selection shortlist - Merge top Lasso drivers, retained PCA components, and the FCI into a final factor set (~8–12 series) per country, saving `outputs/factors_{country}.csv` for downstream modeling.

Volatility & correlation modeling - Fit univariate GARCH(1,1)/GJR models for each factor (`models/garch_{country}_{factor}.pkl`, `outputs/residuals_{country}.csv`) and export standardized residuals for ADCC estimation via R's `rmgarch`. - Import ADCC correlation paths back into Python (CSV/NetCDF), combine with GARCH vols to produce $\Sigma_t = D_t R_t D_t$ covariance matrices for scenario generation.

Scenario engine (multi-mode) - Support four scenario modes: * **Deterministic**: impose predefined factor paths (e.g., +300bp rates, -35% equities) with documented timing/persistence. * **Stochastic Monte Carlo**: draw from MVN/t distributions parameterized by Σ_t , compute VaR/ES, and store reproducible draws. * **Historical replay**: replay known episodes (Lehman, COVID, Eurozone stress) scaled to current volatility/correlation regimes. * **Reverse stress**: optimize minimal factor shocks that achieve defined country loss thresholds via constrained optimization/quadratic programming. - Store scenario metadata per mode so runs remain auditable.

Scenario propagation to targets & banking satellites - Translate factor shocks to macro/mkt targets using Lasso coefficients (decoded from standardized inputs) and produce ΔGDP , ΔCPI , $\Delta\text{spreads}$, etc. - Apply banking satellite formulas (NPL elasticity, sovereign duration loss, funding stress) to compute CET1 pressure and stress metrics per scenario.

Validation & backtesting - Validate deterministic/historical scenarios against historical episodes (2008, 2011, 2020) and confirm MC quantiles align with realized extremes. - Monitor ADCC/GARCH diagnostics (likelihood/AIC, persistence) and ensure scenario propagation outputs contain no NaNs/inf values.

Automation driver (optional future project) - Reserve automation for future sprints; continue manual orchestration while logging dependencies between steps.

Reporting outputs - Produce `outputs/country_report_{country}.pdf`, `outputs/country_representation_{country}.json`, and companion dashboards/notebooks highlighting scenario inputs, factor contributions, and stress metrics.

This scenario-first orientation keeps the engine central and readies the workflow for deterministic, Monte Carlo, historical replay, and reverse stress-testing modes while deferring automatic orchestration until the inputs stabilize.

Immediate next actions (to start the stress-testing programme) 1) Formalize config/series_metadata.yaml and ingest additional IMF/ECB indicators needed for the scenario factor set. 2) Run the Scenario-First workflow end-to-end for Italy, producing outputs/factors_ITA.csv, outputs/feature_contributions_ITA.csv, and a deterministic scenario trace. 3) Pilot the Python→R ADCC bridge with those factors and save Σ_t outputs for the scenario configurations. 4) Draft the first outputs/country_report_ITA.pdf summarizing the scenario diagnostics and outcomes.

Note on structural inputs - The structural data work previously planned remains valuable but optional this sprint. Start with transparent proxies (GDP-scaled exposures, identity mappings) and document any imputations; incorporate the structural theta matrix once curated.

Phase 7: Continuous Improvement and Extensions — Planned/Ongoing

Potential enhancements - Non-linear and state-dependent exposures - Dynamic factor models - Machine learning variants - Additional countries - Sectoral decomposition - Integration with DNSS yield-curve beta models for full yield curve stress testing - Real-time monitoring - Academic write-up - The structural data collection (balance-sheet theta), empirical exposure estimation (regression), and validation/reconciliation remain documented here as ideas for further development. They are valuable follow-ups but will be treated as optional extensions to the stress-testing programme and will be re-activated if/when data procurement and bandwidth allow. - Structural Data Collection (balance-sheet theta): collect sovereign, banking, corporate/household, external sector series and compute a results/structural_theta_matrix.csv. (See previous text for deliverables and tentatively assigned files.) - Empirical Exposure Estimation (regression): align factors and outcomes, run ridge and panel regressions to estimate empirical thetas and time-varying exposures. Outputs would include results/empirical_theta_matrix.csv and model comparison reports. - Validation and Reconciliation: compare structural vs empirical theta, produce results/theta_comparison_report.html and uncertainty estimates. Combine structural and empirical via documented aggregation rules.

Current Status and Next Actions

Date: 2026-01-02

Status summary - Phase 0: Completed — data gathering, merge, QA and crisis coverage prepared - Phase 1: Completed — NSS/DNSS estimation; monthly recommended; daily Kalman+RTS for diagnostics - Phase 2: Completed and validated — DCC-GARCH framework working; crisis heatmaps available - Phase 3: Step 0/1 rerun on 2026-01-02 refreshed metadata guardrails, regenerated analysis_outputs/diagnostics/resampling_log.csv (60 series logged) and global_data_health.json, and reran block coverage (still partial/insufficient for the main Bund/BTP/Bonos spreads) before moving downstream - ADCC vs DCC: Numeric summary available (analysis_outputs/dcc_vs_adcc_gap_summary.csv) - Interview package: Created under interview_package with scripts, Italian summary (Sommaro.md), manifest for images, and README

Immediate next actions 1) Backfill or proxy the missing BTP_Bund_Spread, Bonos_Bund_Spread, and OAT_Bund_Spread series (or formally mark them optional) so country blocks can move out of partial/insufficient states. 2) Keep the Step 3-5 factor prep + Lasso/shortlist workflows aligned with the reviewed metadata/resampling heuristics (≤ 3 drivers, 8–12 shortlist, resampling_log.csv entries) before re-launching the diagnostics/validation sweeps. 3) Bundle the updated metadata/resampling & diagnostics evidence into the next summary for stakeholders so the scenario engine knows the newly

resampled panel has been certified.

Blockers - None identified; Phase 2 requires API credentials or public endpoints for ECB/Eurostat

References and Resources

Key papers 1. Engle (2002): Dynamic Conditional Correlation 2. IMF (2017): Annual Macrofinancial Stress Testing Exercise 3. Adrian et al. (2019): Growth-at-Risk 4. ECB/ESRB (2021): Systemic Risk Survey

Data sources - ECB SDW: <https://sdw.ecb.europa.eu/> - Eurostat: <https://ec.europa.eu/eurostat> - IMF Data: <https://data.imf.org/> - BIS Statistics: <https://www.bis.org/statistics/>

Code libraries - arch, statsmodels, scikit-learn, pymc, dash/streamlit

Notes - This document is a living plan. Update checkboxes and notes as work progresses.